Pools: Fonds as Passive Data Resources

What Is a Fond?

What Is a Fond? I

A **fond** is a versioned, read-only data pool that any project in the repository can consume without modifying. The term evokes the culinary concept of a concentrated stock — a stable base that enriches whatever is built on top of it. Fonds live under the top-level `fonds/<scope>/<name>/` directory, each containing a manifest file (`fonds.yaml`), a `data/` subdirectory, and optional documentation. This architecture separates *data ownership* from *data usage*: research projects in `projects/` are consumers, not producers, of fond data. The separation prevents the accretion of project-specific mutations in shared resources — a recurring source of reproducibility failures in collaborative research software (Wilson et al. 2014).

What Is a Fond? I

The three-layer taxonomy (bibliography, contacts, datasets) maps to the three most common categories of curated research data: citable literature, human collaboration networks, and input/output datasets. Each category carries its own schema enforced by `rules/templates/template_manuscript_rules`. fig. 2 compares the three fond types field-by-field: every fond shares a manifest and a `type` discriminator, but the source-of-truth format, secondary mirror, and deduplication key differ by category — a consequence of each category's data having a naturally different canonical representation (BibTeX for citations, YAML for structured contact records, YAML for dataset metadata).

What Is a Fond? I

Fond Schema Taxonomy: Bibliography × Contacts × Datasets

Field	Bibliography	Contacts	Datasets
Manifest (`fonds.yaml`)	✓	✓	✓
Required `type` field	✓	✓	✓
Primary key	✓	✓	✓
Source-of-truth format	BibTeX	YAML	YAML
Secondary mirror	CSV	JSON	—
Dedup key	cite key	`id`	`id`
Binary data committed	-	-	-

Figure 2: Schema taxonomy comparison across the three fond types. Each fond declares a manifest and a `type` field; source-of-truth format, secondary mirror, and dedup key differ by category.

The Three Template Fonds

The template_bibliography fond is a curated reference library stored in two formats: a BibTeX file (data/references.bib) as source of truth, and a flat CSV export (data/references.csv) for programmatic querying. Deduplication is enforced on the cite key (the primary CSV column). The collection spans foundational machine-learning works — the transformer architecture (Vaswani et al. 2017), early convolutional network research (LeCun et al. 1998), and large-scale language model pre-training (Brown et al. 2020) — alongside software-engineering references on best practices (Wilson et al. 2014) and robust software design (Taschuk and Wilson 2017). In the current integration run, the fond contains **8 entries**.

The bibliography fond illustrates the *registry pattern* (Fowler 2002): a single authoritative list is maintained centrally, and all projects reference it rather than keeping private copies. This guarantees citation consistency across all exemplar manuscripts.

The `template_contacts` fond holds a registry of research collaborators, advisors, and reviewers. Each entry is a YAML object with required fields `id` (a unique slug), `name`, and `email`, plus optional fields `affiliation`, `role`, `orcid`, `website`, and `notes`. The YAML file (`data/contacts.yaml`) is the source of truth; a JSON mirror (`data/contacts.json`) supports consumers that prefer JSON deserialization. Deduplication is enforced on the `id` field at validation time.

The `template_datasets` fond catalogs dataset metadata: provenance, licensing, format, size, access URLs, and research tasks. It intentionally stores *metadata only* — no actual data binaries are committed to the repository. This design aligns with the principle that version control systems should track configuration and metadata rather than large binary artefacts (Kluyver et al. 2016). Dataset entries require `id`, `name`, `version`, and `license` fields. Exemplar entries reference classic benchmarks such as MNIST (introduced in (LeCun et al. 1998)) and large-scale corpora used in language-model research (Brown et al. 2020).

Fond Manifest Contract

Fond Manifest Contract I

Every fond root must contain a `fonds.yaml` manifest with at minimum three fields:

```yaml
type: bibliography   # bibliography | contacts | datasets
description: "Human-readable description of the fond"
version: "1.0"
tags: [curated, exemplar]
```

Fond Manifest Contract I

The `type` field governs which reader function is appropriate and what schema the `data/` directory is expected to follow. The `version` field is incremented whenever the schema changes, enabling consumers to detect and handle schema drift without silent failures.

Fond Reader Module

Fond Reader Module I

The `src/template_pools_rules_tools/tools/fonds_reader.py` module provides three reader functions — one per fond type — plus a convenience aggregator:

```
from template_pools_rules_tools.tools.fonds_reader
import (
    read_bibliography_fond,
    read_contacts_fond,
    read_datasets_fond,
    read_all_fonds,
)

bib      = read_bibliography_fond()    # dict | None
contacts = read_contacts_fond()        # dict | None
datasets = read_datasets_fond()        # dict | None
all_fonds = read_all_fonds()           #
{"bibliography": ..., "contacts": ..., "datasets":
...}
```

Fond Reader Module I

Each reader resolves the repository root from `pathlib.Path(__file__).resolve().parents[6]`, checks that the manifest and data files exist before touching them, and wraps the actual YAML parse in a `try/except (OSError, UnicodeDecodeError, yaml.YAMLError)` block. A missing path or a malformed file both degrade the same way — a logged warning and a `None` return — so the integration pipeline keeps going when a fond has not yet been populated by a parallel agent (Taschuk and Wilson 2017). In the current run, 3 of 3 expected fonds were successfully loaded (see fig. 5).

Resilience by Design

Resilience by Design I

The fond layer enforces resilience at two levels. At the **structural** level, readers tolerate missing fonds entirely. At the **schema** level, the manifest version field allows consumers to check compatibility before processing data. This two-level approach means a fond can evolve its schema without breaking existing consumers that have not yet been updated: the consumer detects the version mismatch and either adapts to it or skips the fond outright, instead of crashing on data it doesn't recognise.

Worked Example: Graceful Degradation in Practice

Worked Example: Graceful Degradation in Practice I

Consider a concrete failure scenario: a parallel automation agent is in the process of authoring `fonds/templates/template_contacts/` and has written `fonds.yaml` but not yet populated `data/contacts.yaml`. A naive reader would raise `FileNotFoundError` the instant `read_contacts_fond()` is called, aborting the entire integration pipeline over one incomplete resource. `read_contacts_fond()` instead:

Worked Example: Graceful Degradation in Practice I

1. Resolves the repository root via `pathlib.Path(__file__).resolve().parents[6]`.
2. Checks `manifest_path.exists()` and `contacts_path.exists()` explicitly before any read; on a missing path, logs `logger.warning("contacts fond: missing %s", p)` and returns `None` immediately — no exception is ever raised for the common case of an in-progress resource.
3. If both paths exist, parses each with `yaml.safe_load()` inside a `try/except (OSError, UnicodeDecodeError, yaml.YAMLError)` block, so a present-but-malformed file degrades the same way as a missing one.
4. `run_integration_demo()` records the reduced count in the summary dict rather than propagating any exception.
5. The manuscript token 3 reflects the reduced count honestly — the pipeline never claims a fond loaded that did not.

Worked Example: Graceful Degradation in Practice I

This sequence is exercised directly by `tests/tools/test_fonds_reader.py::test_missing_data_dir_contacts_returns_none`, which constructs a fond directory with a manifest but no `data/` subdirectory and asserts the reader returns `None` rather than raising. The same existence-check-then-parse pattern repeats identically across `fonds_reader.py`'s three readers, `rules_applier.py`, and `tools_invoker.py`, which is why fig. 8 presents it as one repeated design, not three independent ad-hoc fixes.